TECHNICAL NOTICE

Third-party Runtime Components

MediaPipe provenance, modifications, checksums, licensing, and update procedure.

2026 EDITION / IAIN BENNETT / MIT LICENSE

PowerGlove Vision's original source code and associated documentation are licensed under the repository's MIT License. That license does not replace the licenses or terms that apply to third-party software and model files.

MediaPipe 0.10.18 ARM64 wheel

The UNO Q runtime uses this tracked wheel:

```
python/worker-wheels/mediapipe-0.10.18-cp312-cp312-manylinux2014_aarch64.manylinux_2_17_aarch64.whl
```

Property	Value
Component	MediaPipe 0.10.18 for CPython 3.12, Linux ARM64
Upstream project	<https://github.com/google-ai-edge/mediapipe>
Upstream package	<https://pypi.org/project/mediapipe/0.10.18/>
License	Apache License 2.0
PowerGlove repack SHA-256	`f2617c0960eb35aaa58b76076a9ae629edbf7ec8901d5fab4b046c28d13fbe8d`
Original PyPI wheel SHA-256	`09cbf7dc1f9a2deeaaac687e5f982836623def4cbd3e827d95f86f42450d2dd1`

Modification notice

PowerGlove Vision repackaged the upstream wheel for its headless UNO Q worker. The Python package code and compiled MediaPipe binaries were not modified. The wheel dependency metadata was changed as follows, and its `RECORD` was rebuilt:

- removed the `jax` dependency declaration;
- removed the `jaxlib` dependency declaration;
- replaced `opencv-contrib-python` with `opencv-contrib-python-headless==4.10.0.84`;
- omitted the upstream wheel's empty `mediapipe.libs` directory.

These changes avoid unnecessary JAX installation and GUI OpenCV dependencies on the UNO Q. The repacked wheel retains MediaPipe's Apache 2.0 license at `mediapipe-0.10.18.dist-info/LICENSE`. Do not substitute the upstream wheel without retesting dependency resolution, camera startup, and hand tracking.

Google Hand Landmarker model

PowerGlove Vision uses Google's float16 Hand Landmarker task bundle.

Property	Value
UNO Q runtime path	`data/models/hand_landmarker.task`
Official download	<https://storage.googleapis.com/mediapipe-models/hand_landmarker/hand_landmarker/float16/1/hand_landmarker.task>
SHA-256	`fbc2a30080c3c557093b5ddfc334698132eb341044ccee322ccf8bcf3607cde1`
Size	7,819,105 bytes

The model is not tracked in Git and is not included in the public App Lab ZIP. On first launch, the application downloads it from Google's versioned URL into the private, persistent `data/models/` directory. It refuses to start the vision worker if the SHA-256 checksum differs, reports the problem on the dashboard, and retries without taking the web interface offline. Later launches reuse the verified cached model. Wi-Fi deployments preserve `data/`, so an application update does not download the model again.

`scripts/fetch-runtime-assets.sh` provides an optional manual prefetch using the same URL and checksum. It writes to the same private `data/models/` path and is useful for development or for preparing the app before an offline session.

Google's MediaPipe repository is Apache-2.0 licensed and its official examples direct applications to this model URL. The model download does not place a separate model-specific license file in this repository. The PowerGlove Vision MIT License therefore should not be interpreted as licensing the Google model. The project avoids redistributing the model by having each installation fetch it directly from Google. Anyone who independently bundles or redistributes the model should confirm that the intended distribution complies with Google's applicable model terms.

Updating either component

Treat a wheel or model update as a tested dependency change:

1. Record the official source URL, version, license, size, and SHA-256 here.
2. Update the pinned value in `scripts/fetch-runtime-assets.sh` when changing the model.
3. If repackaging another wheel, record every difference from upstream and retain its license files.
4. Build the App Lab ZIP and confirm it contains one wheel, no model file, and only the root `sketch/` application sketch.
5. Test first-launch download and checksum verification, camera initialization, tracking, the Learn and Debug pages, and controller output on the UNO Q before publishing the package.